- Look for high-fidelity fakes.
- Wrap third-party interfaces.

That's a lot to keep in your head, but we're definitely not asking you to do that! All of these bits of advice stem from one key practice: constructing your test environment carefully. Remember the laboratory metaphor when you're writing your specs, and you should be fine.

Above all else, when you find your test doubles straying from these principles, listen to what they're telling you about your code's design. Rather than looking for a different way to write the test, look for a better way to structure your code. By "better," we don't mean "more testable," although as Michael Feathers explains, good design and testability are often mutually reinforcing.[16] We mean easier to maintain, more flexible, easier to understand, and easier to get right. Justin Searls discusses these trade-offs for test doubles on his "Test Doubles" wiki page and in his SCNA 2012 talk, "To Mock or Not to Mock."[17,18]

Thank you for coming along on this journey with us! For one final time, please join us in the next section for an exercise.

Exercise

This exercise will be a little more open-ended than some of the ones from previous chapters. There's not a single best answer. We're going to prompt your creativity with a few questions, and then turn you loose on the code.

The following Ruby class implements a number-guessing game. Save this code into a new directory as lib/guessing_game.rb:

15-using-test-doubles-effectively/exercises/guessing_game/lib/guessing_game.rb

```
class GuessingGame
  def play
    @number = rand(1..100)
    @guess = nil

    5.downto(1) do |remaining_guesses|
      break if @guess == @number
      puts "Pick a number 1-100 (#{remaining_guesses} guesses left):"
      @guess = gets.to_i
      check_guess
    end

    announce_result
  end
```

16. https://vimeo.com/15007792
17. https://github.com/testdouble/contributing-tests/wiki/Test-Double
18. https://vimeo.com/54045166